

Guía de implementación

HU9 · HU11 · HU12 · HU14

Nanys Care

Aplicación de cuidado infantil — Flutter

Plan técnico para implementar las historias de usuario de **recordatorios automáticos**, **calificación de cuidadores**, **evaluación de tutores** y **recordatorios de citas**, respetando la arquitectura actual del proyecto (modelos + servicios ChangeNotifier + StorageService).

- HU9** Recordatorios automáticos
- HU11** Calificar cuidadores
- HU12** Evaluar tutores
- HU14** Recordatorios de citas

Contenido

1. Estado actual y alcance	1
2. HU11 — Calificar cuidadores	1
2.1. Paso 1 — Crear un ResenaService	1
2.2. Paso 2 — Recalcular el promedio en StorageService	2
2.3. Paso 3 — Registrar el servicio en main.dart	2
2.4. Paso 4 — Usar el servicio en tutor_citas_screen.dart	3
2.5. Paso 5 — Mostrar las reseñas en el perfil del cuidador	3
3. HU12 — Evaluar tutores	3
3.1. Decisión de diseño	3
3.2. Paso 1 — Reusar ResenaService	3
3.3. Paso 2 — Pantalla “Mis notas de tutores”	4
3.4. Paso 3 — Enlazar la pantalla	4
4. HU9 + HU14 — Recordatorios automáticos / de citas	4
4.1. Paso 1 — Crear ReminderService	4
4.2. Paso 2 — Registrar y disparar	5
4.3. Paso 3 (opcional) — HU9 más amplio	6
5. Checklist y pruebas	6

Estado actual y alcance

La guía cubre cuatro historias de usuario. El proyecto ya tiene la base (modelos Resena y Notificacion, almacenamiento local y pantallas por rol), por lo que en varios casos solo hay que completar el flujo.

HU	Título	Estado actual
HU9	Recordatorios automáticos	No existe el motor de recordatorios.
HU11	Calificar cuidadores	Parcial: ya se crea la Resena; falta promedio, evitar duplicados y mostrarlas.
HU12	Evaluar tutores	Parcial: ya existe la “nota privada”; falta pantalla para consultarlas.
HU14	Recordatorios de citas	Depende de HU9.

HU9 + HU14 van juntas

HU9 es el **motor** (servicio que genera recordatorios) y HU14 es su **primer uso** (recordatorios de citas próximas).

Orden recomendado

1. **HU11** — completa lo ya empezado; es la más rápida.
2. **HU12** — reutiliza casi todo de HU11.
3. **HU9 + HU14** — el motor nuevo de recordatorios.

HU11 — Calificar cuidadores

Hoy `tutor_citas_screen.dart` (`_calificar`) ya guarda una Resena, pero *no* actualiza el promedio del cuidador, permite calificar la misma cita varias veces y las reseñas no se muestran.

Paso 1 — Crear un `ResenaService`

Centraliza la lógica para no repetir código entre tutor y cuidador. Crea `lib/services/resena_service.dart`:

```
1 import 'package:flutter/foundation.dart';
2 import '../models/resena.dart';
3 import 'storage_service.dart';
4
5 class ResenaService extends ChangeNotifier {
6   final StorageService _storage;
7   ResenaService(this._storage);
8
9   // Resenas PUBLICAS recibidas por un usuario (para mostrar en su perfil).
10  Future<List<Resena>> getResenasPublicas(String destinatarioId) async {
11    final all = await _storage.getResenas();
12    final list = all
13      .where((r) => r.destinatarioId == destinatarioId && !r.esPrivada)
14      .toList();
15    list.sort((a, b) => b.createdAt.compareTo(a.createdAt));
16    return list;
17  }
18
19  // Notas PRIVADAS escritas por un autor (HU12 / RF15).
20  Future<List<Resena>> getNotasPrivadas(String autorId) async {
21    final all = await _storage.getResenas();
22    final list = all.where((r) => r.autorId == autorId && r.esPrivada).toList();
23    list.sort((a, b) => b.createdAt.compareTo(a.createdAt));
24    return list;
25  }
26
27  // Evita duplicados: el autor ya califico esta cita?
28  Future<bool> yaCalifico(String citaId, String autorId) async {
29    final all = await _storage.getResenas();
30    return all.any((r) => r.citaId == citaId && r.autorId == autorId);
31  }
32
33  Future<void> crearResena(Resena resena) async {
34    await _storage.addResena(resena);
35    if (!resena.esPrivada) {
36      await _storage.calcularCalificacionCuidador(resena.destinatarioId);
37    }
38    notifyListeners();
39  }
40 }
```

Paso 2 — Recalcular el promedio en `StorageService`

Añade este método en `lib/services/storage_service.dart` (sección RESEÑAS):

```
1 // Recalcula y persiste calificacionPromedio segun resenas publicas.
2 Future<void> recalcularCalificacionCuidador(String cuidadorId) async {
3   final resenas = await getResenas();
4   final publicas = resenas
5     .where((r) => r.destinatarioId == cuidadorId && !r.esPrivada)
6     .toList();
7   if (publicas.isEmpty) return;
8
9   final promedio =
10     publicas.fold<double>(0, (s, r) => s + r.calificacion) / publicas.length;
11
12   final perfiles = await getPerfilesCuidador();
13   final idx = perfiles.indexWhere((p) => p.userId == cuidadorId);
14   if (idx < 0) return;
```

```

15
16   final p = perfiles[idx];
17   perfiles[idx] = PerfilCuidador(
18     userId: p.userId, descripcion: p.descripcion,
19     nivelExperiencia: p.nivelExperiencia, certificaciones: p.certificaciones,
20     capacidades: p.capacidades, tarifaPorHora: p.tarifaPorHora,
21     ubicacion: p.ubicacion, diasDisponibles: p.diasDisponibles,
22     horarioDisponible: p.horarioDisponible,
23     calificacionPromedio: double.parse(promedio.toStringAsFixed(1)),
24     totalServicios: p.totalServicios, verificado: p.verificado,
25   );
26   await savePerfilesCuidador(perfiles);
27 }

```

Recomendación

Más limpio: añade un `copyWith` a `PerfilCuidador` (como ya lo tiene `Cita`) y úsalo aquí. Recomendado, opcional.

Paso 3 — Registrar el servicio en `main.dart`

```

1   final resenas = ResenaService(storage);
2   // ...y dentro del MultiProvider:
3   ChangeNotifierProvider<ResenaService>.value(value: resenas),

```

Recuerda agregar el campo y el `required this.resenas` en el constructor de `NanysCareApp`, igual que los demás servicios.

Paso 4 — Usar el servicio en `tutor_citas_screen.dart`

En `_calificar`, valida duplicados antes del diálogo y al enviar usa `crearResena` en vez de `storage.addResena`:

```

1   final resenaSvc = context.read<ResenaService>();
2   if (await resenaSvc.yaCalifico(cita.id, auth.currentUser!.id)) {
3     // Snackbar "Ya calificaste esta cita" y return
4   }
5   // ...al confirmar:
6   await resenaSvc.crearResena(resena); // recalcula el promedio

```

Paso 5 — Mostrar las reseñas en el perfil del cuidador

En `cuidador_detalle_screen.dart`, agrega una sección “Reseñas” (es `StatelessWidget`, usa `FutureBuilder`):

```

1   _SectionTitle('Resenas'),
2   FutureBuilder<List<Resena>>(
3     future: context.read<ResenaService>().getResenasPublicas(user.id),
4     builder: (context, snap) {
5       if (!snap.hasData) return const SizedBox.shrink();
6       final resenas = snap.data!;
7       if (resenas.isEmpty) return const Text('Aun no tiene reseñas');
8       return Column(
9         children: resenas.map((r) => ListTile(
10           leading: const Icon(Icons.star, color: AppColors.warning),
11           title: Text('${r.autorNombre}  ${r.calificacion.toStringAsFixed(1)}'),
12           subtitle: Text(r.comentario),
13         )),
14       );
15     },
16   ),

```

Hecho cuando...

El tutor califica una cita completada, *no* puede repetir, el promedio del cuidador sube y las reseñas aparecen en su perfil.

HU12 — Evaluar tutores

cuidador_solicitudes_screen.dart (_pedirNotaPrivada) ya crea una Resena con esPrivada: true al completar la cita. Falta una pantalla donde el cuidador **consulte** esas evaluaciones.

Decisión de diseño

- **Privada (RF15):** la evaluación del tutor solo la ve quien la escribió. Es lo que ya hace el código; mantenerlo así.
- Si tu HU12 exige que **administrador/supervisor** vean las evaluaciones de tutores, entonces no la marques privada o añade un rol autorizado a leerlas.

Paso 1 — Reusar ResenaService

Cambia storage.addResena(...) por:

```
await context.read<ResenaService>().crearResena(resena); // esPrivada: true
```

y valida duplicados con yaCalifico(cita.id, autorId), igual que en HU11.

Paso 2 — Pantalla “Mis notas de tutores”

Crea lib/screens/cuidador/cuidador_notas_screen.dart:

```
1 class CuidadorNotasScreen extends StatelessWidget {
2   const CuidadorNotasScreen({super.key});
3
4   @override
5   Widget build(BuildContext context) {
6     final user = context.watch<AuthService>().currentUser!;
7     return Scaffold(
8       appBar: AppBar(title: const Text('Mis notas de tutores')),
9       body: FutureBuilder<List<Resena>>(
10        future: context.read<ResenaService>().getNotasPrivadas(user.id),
11        builder: (context, snap) {
12          if (!snap.hasData) {
13            return const Center(child: CircularProgressIndicator());
14          }
15          final notas = snap.data!;
16          if (notas.isEmpty) {
17            return const Center(child: Text('Aun no tienes notas privadas'));
18          }
19          return ListView(
20            padding: const EdgeInsets.all(16),
21            children: notas.map((n) => Card(
22              child: ListTile(
23                leading: const Icon(Icons.lock_outline),
24                title: Text('${n.calificacion.toStringAsFixed(1)} estrellas'),
25                subtitle: Text(n.comentario.isEmpty ? '(sin comentario)' : n.comentario),
26              ),
27            ).toList(),
28          );
29        },
30      ),
31    );
32  }
33 }
```

Paso 3 — Enlazar la pantalla

Agrega un acceso desde `cuidador_home.dart` o `cuidador_perfil_screen.dart` (un `ListTile`/botón con `Navigator.push` a `CuidadorNotasScreen`).

Hecho cuando...

Al completar una cita el cuidador evalúa al tutor y puede volver a ver esas evaluaciones en “Mis notas de tutores”.

HU9 + HU14 — Recordatorios automáticos / de citas

No hay backend ni *cron*, así que el patrón realista es: **revisar las citas al arrancar la app o al refrescar el home y generar notificaciones de tipo recordatorio para las citas próximas que aún no tengan una.**

Paso 1 — Crear `ReminderService`

Crea `lib/services/reminder_service.dart`:

```

1  import '../models/cita.dart';
2  import '../models/notificacion.dart';
3  import 'storage_service.dart';
4  import 'notification_service.dart';
5
6  // HU9/HU14: genera recordatorios automaticos de citas proximas.
7  class ReminderService {
8    final StorageService _storage;
9    final NotificationService _notifications;
10   ReminderService(this._storage, this._notifications);
11
12   Future<void> generarRecordatoriosPendientes() async {
13     final ahora = DateTime.now();
14     final citas = await _storage.getCitas();
15     final existentes = await _storage.getNotificaciones();
16
17     for (final c in citas) {
18       if (c.estado != EstadoCita.aceptada) continue; // solo confirmadas
19       final faltan = c.fechaInicio.difference(ahora);
20       if (faltan.isNegative) continue; // ya paso
21       if (faltan.inHours > 24) continue; // dentro de 24h
22
23       bool yaTiene(String userId) => existentes.any((n) =>
24         n.tipo == TipoNotificacion.recordatorio &&
25         n.usuarioId == userId && n.mensaje.contains(c.id));
26
27       final cuando = _fmt(c.fechaInicio);
28       if (!yaTiene(c.tutorId)) {
29         await _notifications.enviarNotificacion(
30           usuarioId: c.tutorId, titulo: 'Recordatorio de cita',
31           mensaje: 'Tu cita con ${c.cuidadorNombre} es el $cuando. [${c.id}]',
32           tipo: TipoNotificacion.recordatorio);
33       }
34       if (!yaTiene(c.cuidadorId)) {
35         await _notifications.enviarNotificacion(
36           usuarioId: c.cuidadorId, titulo: 'Recordatorio de cita',
37           mensaje: 'Tienes una cita con ${c.tutorNombre} el $cuando. [${c.id}]',
38           tipo: TipoNotificacion.recordatorio);
39       }
40     }
41   }
42
43   String _fmt(DateTime d) =>
44     '${d.day}/${d.month}/${d.year} ${d.hour}:${d.minute}';
45 }

```

Detección de duplicados

El `[$c.id]` dentro del mensaje permite detectar duplicados sin tocar el modelo. Si prefieres algo más limpio, agrega un campo opcional `citaId` a `Notificacion` (actualizando `toJson/fromJson`).

Paso 2 — Registrar y disparar

En `main.dart`:

```
1 final reminders = ReminderService(storage, notifications);
2 await reminders.generarRecordatoriosPendientes(); // al arrancar
```

Vuelve a llamarlo al entrar al home o en el *pull-to-refresh* de `notificaciones_screen.dart` (dentro de `_load`):

```
1 await context.read<ReminderService>().generarRecordatoriosPendientes();
```

Paso 3 (opcional) — HU9 más amplio

El mismo servicio puede generar otros recordatorios:

- recordar al tutor **calificar** una cita completada sin reseña (enlaza con HU11);
- recordar al cuidador **completar** una cita aceptada cuya `fechaFin` ya pasó.

La UI ya está lista: `notificaciones_screen.dart` muestra el ícono de alarma y color naranja para `TipoNotificacion.recordatorio`.

Hecho cuando...

Al abrir la app, las citas aceptadas dentro de las próximas 24 h generan una notificación de recordatorio (sin duplicarse) visible para tutor y cuidador.

Checklist y pruebas**Checklist**

- ☐ `ResenaService` creado y registrado en `main.dart`.
- ☐ `StorageService.recalcularCalificacionCuidador` añadido.
- ☐ HU11: validación de duplicado + promedio actualizado + reseñas visibles.
- ☐ HU12: crearResena con `esPrivada:true` + pantalla “Mis notas”.
- ☐ `ReminderService` creado y registrado.
- ☐ HU9/HU14: recordatorios al arrancar y al refrescar notificaciones.
- ☐ `flutter analyze` sin errores y `flutter run` probado.

Cómo probar rápido (datos demo)

Tutor `carlos@nanyscare.com / 123456`

Cuidadora `maria@nanyscare.com / 123456`

Flujo: el tutor agenda cita → la cuidadora la acepta → la cuidadora la “marca completada” (evalúa al tutor, HU12) → el tutor califica (HU11) → reabrir la app para ver los recordatorios (HU9/HU14). Para reiniciar datos: `StorageService.reset()`.

Nanys Care · Documento técnico interno